Machine Learning Workflow: From Raw Data to Model Evaluation

---

1. Problem Understanding

Whenever you are given a dataset, your goal is to:

1. Load and inspect the data
2. Clean and preprocess it
3. Separate input features (X) and target variable (y)
4. Train a machine learning model
5. Test the model on unseen data
6. Evaluate performance using appropriate metrics
7. Compare multiple models and select the best one

---

2. Dataset Handling

2.1 Loading the Dataset

Most datasets are provided in CSV format.

```
import pandas as pd

df = pd.read_csv("data.csv")
```

Here, `df` is a DataFrame, which is a tabular structure similar to an Excel sheet.

2.2 Understanding the Dataset

Before applying any model, you must understand the structure and contents.

```
df.head()         # View first 5 rows
df.info()         # Data types and null values
df.describe()     # Statistical summary
```

This helps you identify:

- Numerical vs categorical features
- Missing values
- Data distribution

2.3 Handling Missing Values

Check for missing values:

```
df.isnull().sum()
```

Option 1: Fill Missing Values

```
df.fillna(df.mean(), inplace=True)
```

Use this when:

- Missing values are few
- Feature is numerical

Option 2: Drop Missing Rows

```
df.dropna(inplace=True)
```

Use this when:

- Dataset is large
- Missing rows are insignificant

2.4 Handling Categorical Data

Machine learning models require numerical input.

Label Encoding (Ordinal / Binary Categories)

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
```

This converts categories into integers.

One-Hot Encoding (Nominal Categories)

```
df = pd.get_dummies(df, drop_first=True)
```

This creates separate binary columns for each category.

2.5 Splitting Features and Target

Separate input variables (X) and output variable (y).

```
X = df.drop("Price", axis=1)
y = df["Price"]
```

2.6 Feature Scaling

Scaling ensures all features are on the same range.

Important for:

- K-Nearest Neighbors
- Support Vector Machines
- Principal Component Analysis

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

---

3. Train-Test Split

Purpose

To evaluate how the model performs on unseen data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

- 80% data is used for training
- 20% data is used for testing

---

4. Model Implementation

All models in scikit-learn follow the same pattern:

```
model = ModelName()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

---

5. Regression Models (Predict Continuous Values)

Linear Regression

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used when:

- Relationship is approximately linear

Random Forest Regressor

```python
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used when:

- Data has non-linear relationships
- Higher accuracy is required

---

6. Classification Models (Predict Categories)

Logistic Regression

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used for:

- Binary classification
- Linear decision boundary

K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used when:

- Dataset is small
- Patterns are local

Support Vector Machine (SVM)

```
from sklearn.svm import SVC

model = SVC(probability=True)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used when:

- Data is high-dimensional
- Clear margin exists between classes

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Used when:

- Data is complex and non-linear
- Robust performance is needed

---

7. Clustering (Unsupervised Learning)

K-Means Clustering

```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=3)
model.fit(X)

labels = model.predict(X)
```

Used when:

- No target variable exists
- You want to group similar data points

---

8. Dimensionality Reduction

Principal Component Analysis (PCA)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_new = pca.fit_transform(X)
```

Used when:

- Dataset has many features

- You want visualization or noise reduction

---

9. Evaluation Metrics

9.1 Classification Metrics

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
```

Interpretation

- Accuracy: Overall correctness
- Precision: Correct positive predictions out of predicted positives
- Recall: Correct positives out of actual positives
- F1 Score: Balance between precision and recall

Use recall when false negatives are costly (e.g., disease detection).

9.2 Regression Metrics

```
from sklearn.metrics import mean_squared_error, mean_absolute
_error, r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("MSE:", mse)
print("RMSE:", rmse)
```

```
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

Interpretation

- MSE: Penalizes large errors
- RMSE: Error in original scale
- MAE: Average error
- R^2 Score: Proportion of variance explained

9.3 Clustering Metric

```
from sklearn.metrics import silhouette_score

print(silhouette_score(X, labels))
```

Higher score indicates better-defined clusters.

---

10. Model Comparison

Example: Comparing Logistic Regression and Random Forest

```
# Logistic Regression
model1 = LogisticRegression()
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)

# Random Forest
model2 = RandomForestClassifier()
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)

print("Logistic:", accuracy_score(y_test, pred1))
print("Random Forest:", accuracy_score(y_test, pred2))
```

Overfitting Check

```
print("Train Score:", model.score(X_train, y_train))
print("Test Score:", model.score(X_test, y_test))
```

- High train score + low test score indicates overfitting
- Similar scores indicate good generalization

Model Selection Justification

Example:

"Random Forest is preferred because it captures non-linear patterns and reduces overfitting through ensemble learning."

11. Complete End-to-End Pipeline

```
import pandas as pd

# Load dataset
df = pd.read_csv("data.csv")

# Data cleaning
df.fillna(df.mean(), inplace=True)
df = pd.get_dummies(df, drop_first=True)

# Split features and target
X = df.drop("target", axis=1)
y = df["target"]

# Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
# Model training
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
from sklearn.metrics import accuracy_score
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Classification Workflow: Loan Default Prediction Example

1. Problem Understanding

The given problem is to predict whether a customer will default on a loan based on attributes such as age, income, credit score, loan amount, employment status, and number of existing loans.

This is a **binary classification problem**, where:

- Output = 1 → Customer will default
- Output = 0 → Customer will not default

Suitable Algorithms

- Logistic Regression: Works well for linear relationships and provides interpretability
- Random Forest Classifier: Handles non-linear patterns and reduces overfitting

2. Loading and Exploring the Dataset

Code

```
import pandas as pd

df = pd.read_csv("loan_data.csv")

print(df.head())
print(df.info())
print(df.describe())
```

Explanation

- `head()` displays sample rows to understand structure
- `info()` shows data types and missing values
- `describe()` provides statistical summaries

Observations to Make

- Identify missing values
- Identify categorical features (e.g., Employment_Status)
- Identify the target column (assumed as `Default`)

---

3. Data Preprocessing

3.1 Handling Missing Values

```
print(df.isnull().sum())

df.fillna(df.mean(numeric_only=True), inplace=True)
```

Explanation

- Missing values can affect model performance
- Numerical columns are filled using mean values
- For categorical columns, mode can be used if needed

3.2 Encoding Categorical Variables

```
df = pd.get_dummies(df, drop_first=True)
```

Explanation

- Converts categorical variables into numerical format
- `drop_first=True` avoids redundancy (dummy variable trap)

3.3 Splitting Features and Target

```
X = df.drop("Default", axis=1)
y = df["Default"]
```

- `X` contains input features
- `y` contains the target variable

3.4 Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Explanation

- Standardizes features to have mean = 0 and variance = 1
- Important for distance-based and gradient-based models such as Logistic Regression

4. Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Explanation

- Training set is used to learn patterns
- Testing set is used to evaluate performance on unseen data
- `test_size=0.2` means 80% training and 20% testing

5. Model Building

5.1 Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model1 = LogisticRegression()
model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)
```

Explanation

- Models probability of default using a linear decision boundary
- Simple and interpretable

5.2 Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

model2 = RandomForestClassifier()
model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)
```

Explanation

- Ensemble of decision trees
- Captures complex, non-linear relationships
- More robust to noise and overfitting

---

6. Model Evaluation

Import Metrics

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score
```

6.1 Logistic Regression Evaluation

```
print("Logistic Regression:")
print("Accuracy:", accuracy_score(y_test, pred1))
print("Precision:", precision_score(y_test, pred1))
print("Recall:", recall_score(y_test, pred1))
print("F1 Score:", f1_score(y_test, pred1))
```

6.2 Random Forest Evaluation

```
print("Random Forest:")
print("Accuracy:", accuracy_score(y_test, pred2))
```

```
print("Precision:", precision_score(y_test, pred2))
print("Recall:", recall_score(y_test, pred2))
print("F1 Score:", f1_score(y_test, pred2))
```

---

7. Metric Interpretation

In loan default prediction, different metrics have different importance:

- Accuracy: Overall correctness
- Precision: Out of predicted defaulters, how many are correct
- Recall: Out of actual defaulters, how many are detected
- F1 Score: Balance between precision and recall

Important Consideration

In this problem:

- False Negative (predicting non-default when actually default) is highly risky
- Therefore, **Recall is the most critical metric**
- F1 Score is also important for balanced evaluation

---

8. Model Comparison

Example Results

Model	Accuracy	Recall	F1 Score
Logistic Regression	0.82	0.70	0.75
Random Forest	0.88	0.80	0.84

Analysis

- Random Forest has higher recall, meaning it detects more actual defaulters
- Random Forest also has a better F1 Score, indicating better balance
- Logistic Regression performs reasonably but misses more defaulters

---

9. Overfitting Check

```
print("Random Forest Train Accuracy:", model2.score(X_train, y_train))
print("Random Forest Test Accuracy:", model2.score(X_test, y_test))
```

Interpretation

- If training accuracy is significantly higher than testing accuracy, the model is overfitting
- If both are similar, the model generalizes well

---

10. Final Justification

Two models were implemented: Logistic Regression and Random Forest.

Logistic Regression is simple and interpretable, making it suitable as a baseline model. However, it assumes linear relationships between features and the target.

Random Forest, on the other hand, captures complex non-linear relationships and reduces overfitting through ensemble learning.

Based on evaluation metrics, Random Forest achieved higher accuracy, recall, and F1 Score. Since recall is critical in loan default prediction to minimize financial risk, Random Forest is selected as the better-performing model.

---

11. Complete Pipeline (Concise Version)

```
import pandas as pd

# Load dataset
df = pd.read_csv("loan_data.csv")

# Preprocessing
df.fillna(df.mean(numeric_only=True), inplace=True)
```

```python
df = pd.get_dummies(df, drop_first=True)

X = df.drop("Default", axis=1)
y = df["Default"]

from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(X)

# Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2)

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

lr = LogisticRegression().fit(X_train, y_train)
rf = RandomForestClassifier().fit(X_train, y_train)

# Predictions
p1 = lr.predict(X_test)
p2 = rf.predict(X_test)

# Evaluation
from sklearn.metrics import f1_score

print("Logistic Regression F1:", f1_score(y_test, p1))
print("Random Forest F1:", f1_score(y_test, p2))
```

---

12. Key Takeaways

- This is a binary classification problem
- Always preprocess data before modeling

- Use multiple models for comparison
- Choose metrics based on problem context (recall in this case)
- Random Forest is generally preferred for complex, real-world datasets

---

Regression Workflow: Medical Insurance Charges Prediction Example

1. Problem Understanding

The objective is to predict medical insurance charges for individuals based on features such as age, BMI, smoking status, number of children, region, and gender.

This is a **regression problem** because:

- The target variable `charges` is continuous (numerical value)
- The goal is to estimate a real-valued output

Suitable Models

- Linear Regression: Provides a simple baseline and assumes linear relationships
- Random Forest Regressor: Captures non-linear relationships and feature interactions

---

2. Dataset Selection

A commonly used dataset for this problem is the **Medical Cost Personal Dataset** from Kaggle (typically named `insurance.csv`).

Typical features include:

- `age`: Age of the individual

- `bmi`: Body Mass Index
- `smoker`: Smoking status (yes/no)
- `children`: Number of dependents
- `region`: Residential area
- `sex`: Gender
- `charges`: Medical insurance cost (target variable)

---

3. Loading and Exploring the Dataset

Code

```
import pandas as pd

df = pd.read_csv("insurance.csv")

print(df.head())
print(df.info())
print(df.describe())
```

Explanation

- `head()` shows sample data
- `info()` reveals data types and missing values
- `describe()` gives statistical insights

Check for Missing Values

```
print(df.isnull().sum())
```

---

4. Data Preprocessing

4.1 Handling Missing Values

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

Explanation:

- Numerical missing values are replaced with mean
- If categorical values are missing, mode can be used

4.2 Encoding Categorical Variables

Categorical columns include:

- `sex`
- `smoker`
- `region`

```
df = pd.get_dummies(df, drop_first=True)
```

Explanation:

- Converts categorical data into binary columns
- Prevents redundancy using `drop_first=True`

4.3 Splitting Features and Target

```
X = df.drop("charges", axis=1)
y = df["charges"]
```

- `X` contains input features
- `y` contains the target variable

4.4 Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Explanation:

- Standardizes features to a common scale
- Important for Linear Regression optimization

---

5. Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Explanation:

- 80% data is used for training
- 20% is used for testing
- Ensures unbiased evaluation

---

6. Model Building

6.1 Linear Regression

```
from sklearn.linear_model import LinearRegression

model1 = LinearRegression()
model1.fit(X_train, y_train)
```

```
pred1 = model1.predict(X_test)
```

Explanation:

- Assumes linear relationship between inputs and output
- Provides interpretable coefficients

6.2 Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor

model2 = RandomForestRegressor()
model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)
```

Explanation:

- Uses multiple decision trees
- Captures complex patterns
- Handles non-linearity effectively

---

7. Model Evaluation

Import Metrics

```
from sklearn.metrics import mean_squared_error, mean_absolute
_error, r2_score
import numpy as np
```

7.1 Linear Regression Evaluation

```
print("Linear Regression:")
print("MSE:", mean_squared_error(y_test, pred1))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred1)))
print("MAE:", mean_absolute_error(y_test, pred1))
print("R2 Score:", r2_score(y_test, pred1))
```

7.2 Random Forest Evaluation

```
print("Random Forest:")
print("MSE:", mean_squared_error(y_test, pred2))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred2)))
print("MAE:", mean_absolute_error(y_test, pred2))
print("R2 Score:", r2_score(y_test, pred2))
```

---

8. Metric Interpretation

- Mean Squared Error (MSE): Penalizes large errors more heavily
- Root Mean Squared Error (RMSE): Error in same units as target
- Mean Absolute Error (MAE): Average absolute difference
- R^2 Score: Proportion of variance explained by the model

Important Consideration

In this problem:

- Accurate cost estimation is important
- RMSE is preferred because it is interpretable
- R^2 indicates overall model effectiveness

---

9. Model Comparison

Example Results

Model	RMSE	R^2 Score
Linear Regression	6000	0.75
Random Forest	4000	0.88

Analysis

- Random Forest has lower RMSE, meaning predictions are closer to actual values
- Random Forest has higher R^2, meaning it explains more variance
- Linear Regression may underperform due to non-linear relationships

---

10. Overfitting Check

```
print("Random Forest Train R2:", model2.score(X_train, y_trai
n))
print("Random Forest Test R2:", model2.score(X_test, y_test))
```

Interpretation

- If training score is significantly higher than testing score, the model is overfitting
- Similar values indicate good generalization

---

11. Final Justification

Two models were implemented: Linear Regression and Random Forest Regressor.

Linear Regression provides a simple and interpretable baseline but assumes linear relationships between features and target.

Random Forest Regressor captures complex non-linear relationships and interactions among features.

Based on evaluation metrics, Random Forest achieved lower RMSE and higher R^2 score, indicating superior predictive performance. Therefore, Random Forest is selected as the final model.

---

12. Complete Pipeline (Concise Version)

```
import pandas as pd

# Load dataset
df = pd.read_csv("insurance.csv")

# Preprocessing
df = pd.get_dummies(df, drop_first=True)

X = df.drop("charges", axis=1)
y = df["charges"]

from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(X)

# Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2)

# Models
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

lr = LinearRegression().fit(X_train, y_train)
rf = RandomForestRegressor().fit(X_train, y_train)

# Predict
p1 = lr.predict(X_test)
p2 = rf.predict(X_test)

# Evaluate
from sklearn.metrics import r2_score
```

```
print("Linear Regression R2:", r2_score(y_test, p1))
print("Random Forest R2:", r2_score(y_test, p2))
```

13. Key Takeaways

- Continuous target variable indicates a regression problem
- Data preprocessing is essential before modeling
- Linear Regression works well for simple relationships
- Random Forest performs better on complex real-world data
- RMSE and R^2 are key metrics for evaluation

Unsupervised Learning Workflow: K-Means Clustering and PCA

Part 1: K-Means Clustering

1. Problem Understanding

K-Means clustering is an **unsupervised learning algorithm** used to group similar data points into clusters.

- There is **no target variable**
- The goal is to discover hidden patterns or groupings in the data

2. Loading and Exploring the Dataset

```
import pandas as pd

df = pd.read_csv("data.csv")
```

```
print(df.head())
print(df.info())
```

Explanation

- `head()` displays sample data
- `info()` shows data types and missing values

Observations

- Dataset should primarily contain numerical features
- There should be no predefined target column

---

3. Data Preprocessing

3.1 Handling Missing Values

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

Explanation:

- Missing numerical values are replaced with the mean
- Ensures no interruptions in distance calculations

3.2 Feature Selection

```
X = df
```

Explanation:

- All columns are treated as input features since there is no target

---

3.3 Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Explanation:

- K-Means uses distance-based calculations
- Scaling ensures all features contribute equally

---

4. Choosing the Number of Clusters (K)

Elbow Method

```
from sklearn.cluster import KMeans

inertia = []

for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)
```

Plotting

```
import matplotlib.pyplot as plt

plt.plot(range(1, 11), inertia)
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

Interpretation

- Inertia measures within-cluster distance
- The “elbow point” indicates the optimal number of clusters
- After this point, improvement slows down

---

5. Model Training

```
model = KMeans(n_clusters=3, random_state=42)
model.fit(X_scaled)

labels = model.predict(X_scaled)
```

Explanation:

- The model assigns each data point to a cluster
- `labels` contains cluster IDs

---

6. Model Evaluation

Silhouette Score

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)
```

Interpretation

- Value close to 1 → well-separated clusters
- Value close to 0 → overlapping clusters
- Negative value → poor clustering

---

7. Final Output

```
df["Cluster"] = labels
print(df.head())
```

Explanation:

- Adds cluster labels to the dataset for interpretation

8. Final Justification

K-Means clustering was applied to group similar data points.

Data was standardized to ensure fair distance-based calculations.

The optimal number of clusters was selected using the Elbow Method.

The clustering quality was evaluated using the Silhouette Score.

9. Key Takeaways

- K-Means requires feature scaling
- The number of clusters must be chosen carefully
- It is sensitive to outliers
- Evaluation is done using Silhouette Score

Part 2: Principal Component Analysis (PCA)

1. Problem Understanding

Principal Component Analysis (PCA) is a **dimensionality reduction technique**.

- Used when the dataset has many features
- Reduces dimensionality while preserving maximum variance

2. Loading the Dataset

```python
import pandas as pd

df = pd.read_csv("data.csv")
```

3. Data Preprocessing

3.1 Handling Missing Values

```python
df.fillna(df.mean(numeric_only=True), inplace=True)
```

3.2 Feature Selection

```python
X = df
```

3.3 Feature Scaling (Mandatory)

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Explanation:

- PCA is sensitive to feature scale
- Standardization ensures fair contribution

4. Applying PCA

```python
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

Explanation:

- Reduces dataset to 2 principal components
- These components capture maximum variance

---

5. Explained Variance

```
print(pca.explained_variance_ratio_)
print(sum(pca.explained_variance_ratio_))
```

Interpretation

- Each value represents variance captured by each component
- Sum indicates total retained information

Example:

- `[0.60, 0.25]` → 85% of variance retained

---

6. Transformed Dataset

```
df_pca = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
print(df_pca.head())
```

Explanation:

- Original features are replaced with principal components

---

7. Using PCA with a Model (Optional)

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier()
model.fit(X_pca, y)
```

Explanation:

- PCA-reduced data can be used to train models
- Reduces computation and noise

---

8. Final Justification

PCA was applied to reduce the number of features while retaining maximum variance.

The data was standardized prior to applying PCA.

Principal components were selected based on explained variance ratio.

This improves computational efficiency and may enhance model performance.

---

9. Key Takeaways

- PCA reduces dimensionality
- Standardization is mandatory
- It transforms features rather than selecting them
- Evaluation is based on explained variance

K-Means vs PCA (Conceptual Comparison)

Aspect	K-Means Clustering	PCA
Type	Unsupervised learning	Dimensionality reduction
Objective	Group similar data	Reduce number of features
Output	Cluster labels	New transformed features
Requires labels	No	No
Evaluation	Silhouette Score	Explained variance ratio

Final Understanding Strategy

- Use K-Means when the goal is grouping or segmentation
- Use PCA when the goal is reducing dimensionality or improving efficiency
- PCA can also be used before clustering or classification